CS3 SG7 Class Relationship Challenge

Student / Pair Names:
1. Giada T. Paracha
2. Samantha Ofelia Marie P. Galvez
Section: Beryllium

HOW TO USE THIS DOC/INSTRUCTIONS

Type your answers directly under each question. Do not delete the questions or instructions. For the UML diagram, either:
• use Insert > Drawing > New and draw your UML inside this document; OR
• draw it neatly on paper, take a clear photo, and insert the photo under the UML section.
• Write your Python program in VS Code. Submit the .py file separately as instructed. In this Google Doc, paste only the required code evidence.
• Submit this document (in PDF form) and the Python file in Khub.

PART 1 – SELF-STUDY

Use only the SG7 STUDENT Study Guide as your learning reference. Study these topics before answering: **Inheritance, super(), Composition, Aggregation, Dependency, and their UML symbols.**

A. LEARNING TABLE

1. Inheritance
Meaning:
A relationship where a child class inherits attributes and methods from a parent class.

Code clue:
Extends

UML clue:
A solid line with a hollow triangle pointing toward the parent class.

Your own example:
A Dog class inherits from an Animal class, so it can use the methods and attributes of Animal

2. Composition

Meaning:
A strong whole-part relationship where the part depends on the whole to exist.

Where is the part object normally created?
Inside the whole object, usually when the whole is created.

UML clue:
A solid line with a filled diamond pointing toward the whole class.

Your own example:
A House contains Rooms. If the house is destroyed, its rooms as parts of that house cease to exist in the model.

3. Aggregation
Meaning:
A weak whole-part relationship where the part can exist independently of the whole.
How is the related object usually provided to the class?
It is usually passed to the class from outside, such as through a constructor or a setter.
UML clue:
A solid line with a hollow diamond pointing toward the whole class.

Your own example:
A Team has Players, but the players can still exist even if the team is disbanded.

4. Dependency
Meaning:
A relationship where one class temporarily uses or depends on another class to perform a task.

How is the other object used?
It is used temporarily, such as through a method parameter or a local variable.

UML clue:
A dashed arrow pointing toward the class being used.

Your own example:
A Student class uses a Calculator object to solve a physics problem.

B. CHECK YOUR UNDERSTANDING

1. What is the main difference between composition and aggregation?
[Type your answer here]

2. What does super().__init__() allow a child class to do?

[Type your answer here]

3. Which relationship is described by “is-a”?
[Type your answer here]

4. Which relationship is described by “uses-a”?
[Type your answer here]

5. In composition, where is the part object normally created?
[Type your answer here]

6. In aggregation, can the part object still exist without the whole? Explain briefly.
[Type your answer here]

SELF-STUDY CHECKPOINT: You should be able to explain all four relationships in your own words without copying a definition word-for-word.

PART 2 – PAIR ACTIVITY

CLASS RELATIONSHIP CHALLENGE

Scenario: Campus Delivery Robot System

Read the scenario carefully:

• A general Machine stores a serial_number and has a method show_info().
• A DeliveryRobot is a type of Machine. It also stores cargo_capacity.
• Each DeliveryRobot is created with its own ControlBoard. In this system, the ControlBoard is created inside the DeliveryRobot and belongs only to that robot.
• A RoboticsRoom can be assigned to an already existing DeliveryRobot. The robot must still be usable even if the RoboticsRoom object is removed.
• A Technician temporarily uses a DiagnosticTool when checking a robot. The Technician does not permanently store or own the DiagnosticTool.

STEP 1 – IDENTIFY THE RELATIONSHIPS

1. DeliveryRobot → Machine
Relationship: Inheritance
Reason:
DeliveryRobot is a type of Machine, so it inherits the attributes and methods of Machine.

2. DeliveryRobot → ControlBoard
Relationship: Composition
Reason:
The ControlBoard is created inside the DeliveryRobot and belongs only to that robot in this system.

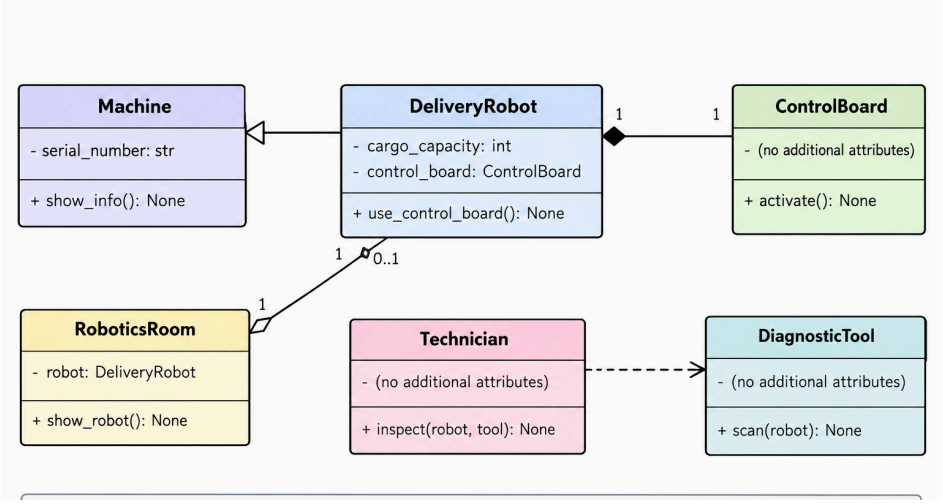
3. RoboticsRoom → DeliveryRobot
Relationship: Aggregation
Reason:
The ControlBoard is created inside the DeliveryRobot and belongs only to that robot in this system.

4. Technician → DiagnosticTool
Relationship: Dependency
Reason:
The ControlBoard is created inside the DeliveryRobot and belongs only to that robot in this system.

STEP 2 – UML DIAGRAM

Your UML must include:
• Machine
• DeliveryRobot
• ControlBoard
• RoboticsRoom
• Technician
• DiagnosticTool
• required attributes and methods
• correct advanced UML relationship symbols

INSERT YOUR UML DIAGRAM BELOW:



STEP 3 – BUILD THE PYTHON PROGRAM

Minimum requirements:

□ The machine has serial_number and show_info().
□ DeliveryRobot inherits from Machine and adds cargo_capacity.
□ DeliveryRobot uses super().__init__() for the parent setup.
□ ControlBoard is created inside DeliveryRobot.__init__().
□ RoboticsRoom receives an already-created DeliveryRobot and stores it.
□ DiagnosticTool has a method such as scan(robot).
□ Technician has a method such as inspect(robot, tool) that temporarily uses DiagnosticTool.
□ Create objects and call methods to demonstrate all four relationships.

REQUIRED TEST SEQUENCE

1. Create one DeliveryRobot.
2. Call the inherited show_info().
3. Call a ControlBoard method through the DeliveryRobot.
4. Create a RoboticsRoom and give it the existing DeliveryRobot.
5. Delete/remove the RoboticsRoom, then show that the DeliveryRobot can still be used.
6. Create a DiagnosticTool and a Technician.
7. Make the Technician inspect the robot using the DiagnosticTool.

STEP 4 – CODE EVIDENCE

Paste only the exact line or short block from YOUR program that proves each relationship.

1. **INHERITANCE**
   Code:
   class DeliveryRobot(Machine):
   What does this line prove?
   It proves inheritance because DeliveryRobot is defined as a subclass of Machine, inheriting its attributes and methods.

2. **COMPOSITION**
   Code:
   self.control_board = ControlBoard()

   What does this line prove?
   It proves composition because the ControlBoard instance is created directly inside the DeliveryRobot constructor, meaning its lifecycle is tied to the robot.

3. **AGGREGATION**
   Code:

   ```
   class RoboticsRoom:
       def __init__(self, robot):
           self.robot = robot
   ```

   What does this line prove?
   It proves aggregation because the RoboticsRoom receives an existing robot and stores a reference to it.

4. **DEPENDENCY**
   Code:

   ```
   def scan(self, robot):
   ```

What does this line prove?
It proves dependency because DiagnosticTool uses DeliveryRobot temporarily as a method parameter without permanently owning it.

STEP 5 – PREDICT BEFORE RUNNING

1. **What attribute from Machine should a DeliveryRobot still have?**
   serial_number

2. **If RoboticsRoom is deleted, should the DeliveryRobot still be usable? Why?**
   Yes, because the relationship is aggregation, meaning the room exists independently of the robot.

3. **Does Technician permanently store DiagnosticTool? Why or why not?**
   No, because the DiagnosticTool is passed temporarily into the technician's inspect() method as a dependency rather than stored as an attribute.

4. **Where should the ControlBoard object be created?**
   Inside the DeliveryRobot.__init__() constructor method.

PART 3 – INDIVIDUAL EXIT TICKET

IMPORTANT:
Each partner must answer this section independently. Put your name before your answers.

STUDENT NAME: Giada T. Paracha

1. Why is DeliveryRobot → Machine inheritance?
DeliveryRobot is a type of Machine. It can inherit the serial number and show_info() method from Machine.

2. Why is DeliveryRobot → ControlBoard composition in this system?
The ControlBoard is created inside the DeliveryRobot and belongs only to that robot in this system.

3. Why is RoboticsRoom → DeliveryRobot aggregation instead of composition?
The robot is created separately and given to the RoboticsRoom. It can still exist and work even if the room is removed.

4. Why is Technician → DiagnosticTool dependency?
The robot is created separately and given to the RoboticsRoom. It can still exist and work even if the room is removed.

5. What problem could occur if DeliveryRobot does not call super().__init__() but later tries to use serial_number?
The robot is created separately and given to the RoboticsRoom. It can still exist and work even if the room is removed.

SUBMISSION CHECKLIST

□ Completed self-study answers
□ Relationship identification
□ UML diagram inserted
□ Working .py file submitted separately
□ Code Evidence section completed
□ Individual Exit Ticket completed by each student

SCORING – 50 POINTS

Self-study notes – 8 points
Relationship identification and reasoning – 8 points
UML diagram – 10 points
Python implementation – 14 points
Program testing and code evidence – 5 points
Individual explanation – 5 points

TOTAL: 50 POINTS